

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 6
«Динамическое программирование»

Выполнил работу
Заборщиков Артём
Академическая группа №J3110
Принято
Ментор, Владислав Вершинин

Санкт-Петербург

2024

Отчёт по лабораторной работе № 5	1
Подсчёт по памяти	3
Подсчёт асимптотики	3
Подтверждение решения	3
Обоснование использование динамического программирования	3
Исходный код	3

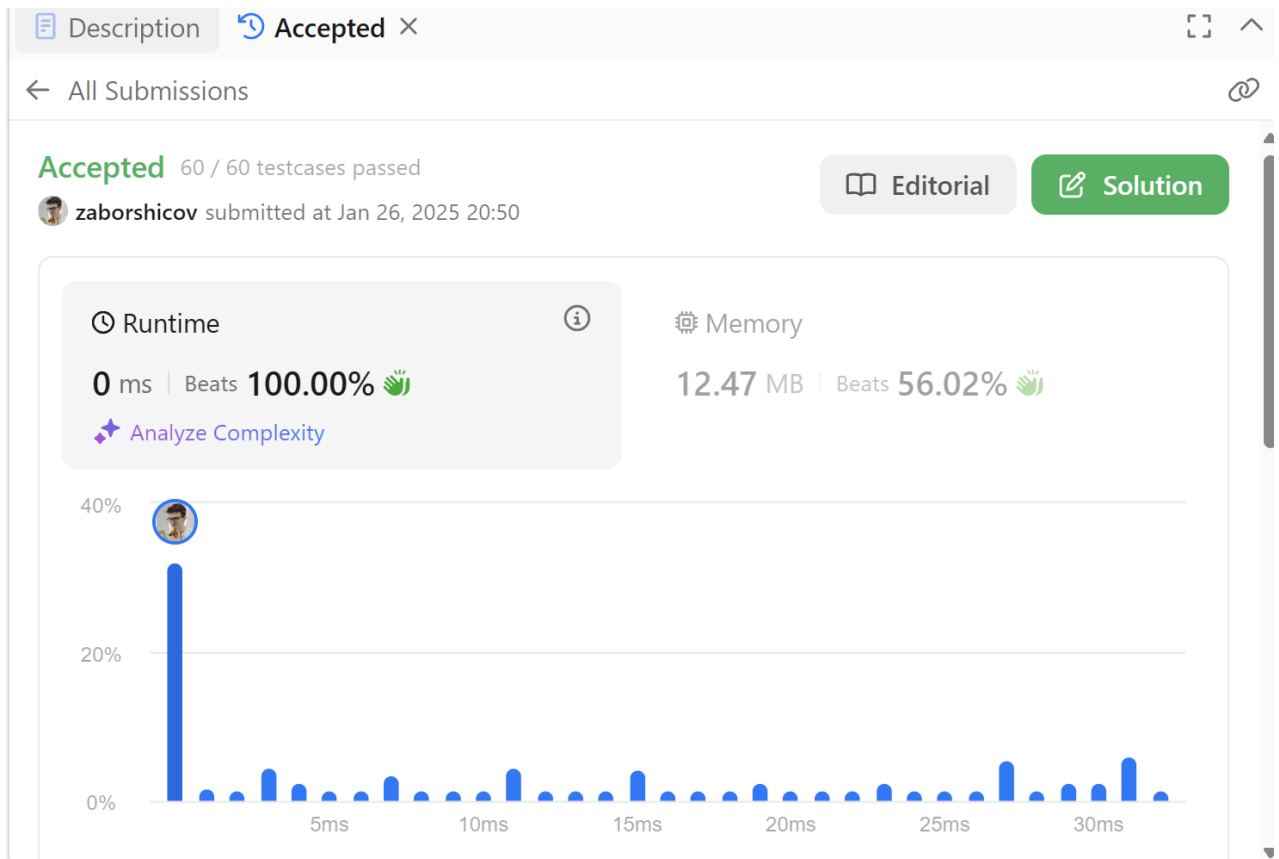
Подсчёт по памяти

$O(n)$ - создаётся несколько массивов с суммарным количеством элементов равным количеству элементов в исходном массиве.

Подсчёт асимптотики

Используется полный проход через массив для разделения элементов на отрицательные и неотрицательные, сложность – $O(n)$. Также оба массива сортируются, сложность – $O(n \log n)$. Далее идёт отбор отрицательных элементов, которые увеличат суммарный **satisfaction**, сложность $O(n)$. Итоговая сложность $O(n \log n)$.

Подтверждение решения



Обоснование использование динамического программирования

Мы один раз решаем задачу о сумме всех элементов которые могут дать какой либо прирост показателя, а потом просто решаем простую задачу о подсчете прироста за добавление элемент используя при этом рекурренту.

Исходный код

```

#include <vector>
#include <algorithm>

class Solution {
public:
    int maxSatisfaction(const vector<int>& satisfaction) {
        std::vector<int> positive;
        std::vector<int> negative;

        for (int elem : satisfaction) {
            if (elem >= 0) {
                positive.push_back(elem);
            } else {
                negative.push_back(elem);
            }
        }

        std::sort(positive.begin(), positive.end());
        std::sort(negative.rbegin(), negative.rend());

        int elem_sum = 0;
        int positive_sum = 0;

        for (int i = 0; i < positive.size(); i++) {
            elem_sum += positive[i] * (i + 1);
            positive_sum += positive[i];
        }

        int bias = 0;
        int elem_sum_neg = 0;

        for (int elem : negative) {
            elem_sum_neg += elem;
            if (positive_sum > -1 * elem_sum_neg) {
                bias += (elem_sum_neg + positive_sum);
            } else {
                break;
            }
        }

        return elem_sum + bias;
    }
};

```

